AIMS — Class Design Specification

Use Case: Place Order

Lab 06 — Class Design | ISD.20252

SOICT – HUST | Lecturer: NGUYEN Thi Thu Trang

Overview

This document provides the detailed class design specification for the Place Order use case in AIMS (An Internet Media Store). It covers all boundary, control, and entity classes identified in the analysis class diagram and sequence diagram for this use case.

Classes covered in this document:

1. Boundary
    1.1. CartScreen
    1.2. DeliveryInfoScreen
    1.3. InvoiceScreen
    1.4. PlaceOrderSuccessfulScreen
    1.5. CreateProductScreen
    1.6. UpdateProductScreen
    1.7. DeleteProductScreen
    1.8. VietQRPaymentScreen
    1.9. CreditCardPaymentForm
    1.10. VietQRBoundary
    1.11. PayPalBoundary
2. Controller
    2.1. PlaceOrderController
    2.2. ProductController
    2.3. PayOrderController
    2.4. PayThroughVietQRController
    2.5. PayThroughCreditCardController
3. Entity
    3.1. CartItem
    3.2. Cart
    3.3. Product
    3.4. ProductHistory
    3.5. Order*
    3.6. Invoice*
    3.7. CreditCard*
    3.8. DeliveryInfo
    3.9. PaymentTransaction*

1.1 CartScreen

Stereotype: <<boundary>>

Displays cart contents and initiates the place order flow. Communicates stock warnings to the customer.

1.1.1 Attributes

#	Name	Type	Default	Visibility	Description
1	cart	Cart	-	-	The current cart object bound to this screen
2	totalExclVAT	double	0	-	Subtotal of all cart items, excluding 10% VAT
3	isLoading	boolean	false	-	Controls loading spinner during async order submission
4	insufficientStockItem	Map<String,int>	new Map()	-	Maps productId with insufficient stock

1.1.2 Operations

#	Name	Return Type	Vis.	Details
1	askToPlaceOrder()	void	+	Triggered when customer clicks "Place Order". Validates cart is non-empty, then calls PlaceOrderController.placeOrder(cart).
2	showInvalidQuantityException(items: Map<string,number>)	void	+	Displays inline error banners for each product that has insufficient stock, showing the available quantity for each.
3	displayCart(cart: Cart)	void	+	Renders the cart product list with quantities, unit prices, and subtotal; called on component initialization.
4	updateTotalDisplay()	void	-	Recalculates and re-renders the subtotal (excl. VAT) whenever cart contents change.

1.1.3 Method — askToPlaceOrder()

1. If cart.items is empty, show "Cart is empty" warning and return.
2. Set isLoading = true; call PlaceOrderController.placeOrder(cart).
3. If InvalidQuantityException is thrown, call showInvalidQuantityException(exception.items).
4. On success, navigate to DeliveryInfoScreen.
5. Set isLoading = false in both success and error paths.

1.2 DeliveryInfoScreen

Stereotype: <<boundary>>

Renders the delivery information form. Recalculates shipping fee on address change. Customers still see cart summary alongside the form.

1.2.1 Attributes

#	Name	Type	Default	Visibility	Description
1	deliveryInfo	DeliveryInfo	new DeliveryInfo()	-	Two-way bound form model containing delivery fields
2	shippingFee	double	0.0	-	Calculated shipping fee displayed to customer
3	cartSummary	CartItem[]	[]	-	Cart items shown alongside form per the problem statement requirement
4	validationErrors	Map<String, String>	{}	-	Maps field name → error message for inline validation feedback
5	isSubmitting	Invoice	false	-	Disables submit button during async submission

1.2.2 Operations

#	Name	Return Type	Vis.	Details
1	showDeliveryForm()	void	+	Renders the delivery form UI with all required fields; called by controller after order is created.
2	setDeliveryInfo(deliveryInfo: DeliveryInfo)	void	+	Handles real-time customer input; updates the deliveryInfo model via two-way binding.
3	calculateShippingFee()	void	+	Adds error message to validationErrors[field] and highlights the invalid form control.
4	saveOrder(invoice: Invoice)	void	+	Called whenever province/address changes. Triggers PlaceOrderController.calculateShippingFee() and refreshes the displayed fee.
5	submitDeliveryInfo()	void	+	Validates the form locally; if valid, calls PlaceOrderController.submitDeliveryInfo(deliveryInfo); otherwise surfaces errors.
6	updateShippingFeeDisplay(fee: double)	void	-	Updates the shippingFee property and triggers re-render of the fee section.

1.2.3 Method — submitDeliveryInfo()

1. Clear validationErrors; set isSubmitting = true.
2. Call PlaceOrderController.checkDeliveryInfoValidity(deliveryInfo).
3. If invalid, call showInvalidDeliveryInfoException(field, message) for each error; set isSubmitting = false; return.
4. Call PlaceOrderController.submitDeliveryInfo(deliveryInfo); navigate to InvoiceScreen.

1.3 InvoiceScreen

Stereotype: <<boundary>>

Displays the full invoice with all costs. Allows customer to choose a payment method (VietQR default, PayPal alternative) and initiate payment.

1.3.1 Attributes

#	Name	Type	Default	Visibility	Description
1	invoice	Invoice		-	The invoice object to render
2	order	Order		-	The current order being paid
3	selectedPaymentMethod	PaymentMethod	VIETQR	-	Current selected payment method
4	qrCodeData	String	null	-	QR code image data returned from VIETQR gateway
5	isProcessingPayment	boolean	false	-	Shows loading state while awaiting payment gateway response

0.0.2 Operations

#	Name	Return Type	Vis.	Details
1	showInvoice()	void	+	Renders invoice details: product list, quantities, unit prices, total excl. VAT, total incl. VAT, delivery fee, and total amount to pay.
2	askToPay()	void	+	Triggered by customer clicking "Pay". Sets isProcessingPayment = true and calls PlaceOrderController.payOrder(order).
3	switchPaymentMethod(method: PaymentMethod)	void	+	Updates selectedPaymentMethod; resets qrCodeData if switching to PayPal; updates the payment UI section.
4	displayQRCode(qrCode: string)	void	+	Sets qrCodeData and renders the VietQR QR code image for the customer to scan.

1.4 SuccessfulScreen

Stereotype: <<boundary>>

Displays order confirmation with customer info, shipping address, and payment transaction details after successful payment.

1.4.1 Attributes

#	Name	Type	Default	Visibility	Description
1	order	Order	-	-	Successfully placed order; source of all displayed info
2	paymentTransaction	PaymentTransaction	-	-	Transaction details (ID, method, datetime, amount)

1.4.2 Operations

#	Name	Return Type	Vis.	Details
1	showSuccessScreen(order: Order)	void	+	Shows customer name, phone, province, shipping address, total amount, transaction ID, transaction content, and transaction datetime.
2	viewOrderDetail(orderId: String)	void	+	Navigates to the order detail page (also accessible via the email link sent to customer)

1.5 CreateProductScreen

Stereotype: <<boundary>>

Represents the screen where the product manager starts the Create Product use case, enters product information, and receives success or validation feedback from the system.

1.5.1 Attributes

#	Name	Type	Default	Visibility	Description
1	currentProductInfo	ProductInfo	null	-	Product information currently entered on the create screen.
2	isCreated	boolean	false	-	Indicates whether the product has been created successfully.

1..2 Operations

#	Name	Return Type	Vis.	Details
1	requestToCreateProduct()	void	+	Triggered when the product manager starts the create product process. Collects product data and sends it to ProductController for processing.
2	showCreateSuccess()	void	+	Displays a success message after the product has been created successfully.
3	displayInvalidNotification(msg: String)	void	+	Displays validation or processing error messages on the screen. **Parameters:** • msg: String — the message to be shown to the product manager.

1.5.3 Method

requestToCreateProduct()

• What: Initiates the create product flow.
• How:
    o Collects product data from ProductInputForm
    o Stores the input into currentProductInfo
    o Calls ProductController.createProduct(currentProductInfo)
    o If the action succeeds, calls showCreateSuccess()
    o If validation fails, calls displayInvalidNotification(msg)
• Use attributes: currentProductInfo, isCreated
• Use relationships: interacts with ProductController

1.6 UpdateProductScreen

Stereotype: <<boundary>>

Represents the screen used by the product manager to update an existing product and receive confirmation or validation feedback from the system.

1.6.1 Attributes

#	Name	Type	Default	Visibility	Description
1	selectedProduct	Product	null	-	The product selected for update.
2	updatedInfo	ProductInfo	null	-	The updated product information entered by the product manager.
3	isUpdated	boolean	false	-	Indicates whether the update action completed successfully.

1.6.2 Operations

#	Name	Return Type	Vis.	Details
1	requestToUpdateProduct()	void	+	Triggered when the product manager starts the update product process. Sends the selected product and updated information to ProductController.
2	displaySuccessConfirmation()	void	+	Displays a confirmation message after the product has been updated successfully.
3	displayInvalidNotification(msg: String)	void	+	Displays validation or processing error messages. **Parameters:** • msg: String — the message to be shown to the product manager.

1.6.3 Method

requestToUpdateProduct()

• What: Initiates the update product flow.
• How:
    o Receives the selected product from ProductList
    o Receives updated data from ProductUpdateForm
    o Stores the selected product in selectedProduct and the new data in updatedInfo
    o Calls ProductController.updateProduct(selectedProduct, updatedInfo)
    o If successful, calls displaySuccessConfirmation()
    o Otherwise, calls displayInvalidNotification(msg)
• Use attributes: selectedProduct, updatedInfo, isUpdated
• Use relationships: interacts with ProductController

1.7 DeleteProductScreen

Stereotype: <<boundary>>

Represents the screen used by the product manager to select products for deletion and display the result of the delete or deactivate action.

1.7.1 Attributes

#	Name	Type	Default	Visibility	Description
1	selectedProducts	String	null	-	The selected product or group of products submitted for deletion.
2	deleteResult	boolean	false	-	Indicates whether the delete/deactivate action completed successfully.

1.7.2 Operations

#	Name	Return Type	Vis.	Details
1	requestToDeleteProducts()	void	+	Triggered when the product manager initiates the delete product process. Sends the selected products to ProductController.
2	displayDeletionResult()	void	+	Displays the final result of the delete or deactivate operation.
3	displayInvalidNotification(msg: String)	void	+	Displays validation or business-rule error messages. Parameters: • msg: String — the message to be shown to the product manager.

1.7.3 Method

requestToDeleteProducts()

• What: Initiates the delete product flow.
• How:
o Receives selected products from ProductSelection
o Stores them in selectedProducts
o Calls ProductController.deleteProduct(...) for each selected product or for the target product
o If stock quantity is zero, the product is deleted
o If stock quantity is greater than zero, the product is deactivated instead
o Calls displayDeletionResult() or displayInvalidNotification(msg) depending on result
• Use attributes: selectedProducts, deleteResult
• Use relationships: interacts with ProductController

1.8 VietQRPaymentScreen

Stereotype: <<boundary>>
The User Interface displaying the QR code alongside a waiting spinner for the customer to scan on their phone.

1.8.1 Attributes

#	Name	Type	Default	Visibility	Description
1	currentInvoice	Invoice	null	-	The current displayed Invoice data model.
2	qrCodeData	String	""	-	Base64 URI String for the <img> tag.
3	isPaymentComplete	boolean	false	-	Status flag indicating if payment has completed.
4	paymentStatus	String	"waiting"	-	Local status tracking: waiting / processing / success / failed.

1.8.2 Operations

#	Name	Return Type	Vis.	Details
1	displayQRCode(invoice: Invoice, qrCode: String)	void	+	Updates the screen to render the invoice details and the Base64 QR image. Parameters: - invoice: Invoice — invoice data to display. - qrCode: String — Base64-encoded QR image string.
2	showPaymentProcessing()	void	+	Disables interactions and shows a loading spinner indicating transaction processing (waiting for VietQR callback).
3	showPaymentResult(success: boolean)	void	+	Renders the success or failure UI sequence. Parameters: - success: boolean — whether payment succeeded.
4	showPaymentError(errorMsg: String)	void	+	Displays an error message on the UI. Parameters: - errorMsg: String — the error message to display.

1.9 CreditCardPaymentForm

Stereotype: <<*boundary*>> (UI)
The UI form (Angular Component) prompting the Customer to input their Credit Card information. Maps to credit-card-payment-form.component.ts.

1.9.1 Attributes

#	Name	Type	Default	Vis.	Description
1	currentInvoice	Invoice	null	-	The current active Invoice data model.
2	cardNumberInput	String	""	-	TextField for the 16-digit card number.
3	expiryInput	String	""	-	TextField for MM/YY expiry format.
4	cvvInput	String	""	-	TextField for the 3-4 digit security code.
5	cardHolderInput	String	""	-	TextField for the capitalized cardholder name.
6	isProcessing	boolean	false	-	Disables the submit button when set to true.
7	paymentStatus	String	"idle"	-	Component state: idle / processing / success / failed.

1.9.2 Operations

#	Name	Return Type	Vis.	Details
1	displayPaymentForm(invoice: Invoice)	void	+	Initializes and renders the card input form with the invoice summary. Parameters: • invoice: Invoice — invoice data to display alongside the form.
2	submitCardInfo(cardInfo: CreditCard)	void	+	Captures the user submit action and forwards card data to the Controller. Parameters: • cardInfo: CreditCard — constructed from form field values.
3	showPaymentProcessing()	void	+	Disables all form inputs and shows a loading spinner.
4	showPaymentResult(success: boolean)	void	+	Navigates to success screen or shows failure alert. Parameters: • success: boolean — whether payment succeeded.
5	showPaymentError(errorMsg: String)	void	+	Displays a popup error for invalid card format or API errors. Parameters: • errorMsg: String — the error message to display.

6	validateFormInput()	boolean	+	Performs client-side (front-end regex) validation on all form fields before submission. Method: Checks cardNumberInput matches 16-digit pattern. Checks expiryInput matches MM/YY format. Checks cvvInput matches 3-4 digit pattern. Checks cardHolderInput is not empty. Returns true if all checks pass.
7	getPaymentStatus()	String	+	Component getter used for front-end data binding. Returns current paymentStatus.

1.10 VietQRBoundary

Stereotype: <<*boundary*>> (System Interface)
System Interface responsible for external VietQR API integration. Authentication via API Key. Maps to NestJS Provider vietqr.provider.ts.

1.10.1 Attributes

#	Name	Type	Default	Vis.	Description
1	apiBaseUrl	String	"https://api.vietqr.vn"	-	Base endpoint URL for VietQR API.
2	clientId	String	env.VIETQR_CLIENT_ID	-	Client ID from environment variable.
3	clientSecret	String	env.VIETQR_CLIENT_SECRET	-	API Key Secret from environment variable.
4	currentToken	String	null	-	Cached access token.
5	tokenExpiry	Date	null	-	Expiry time of the cached token.

1.10.2 Operations

#	Name	Return Type	Vis.	Details
1	getAccessToken()	String	+	Manages token lifecycle; requests a new token if the cached one is expired. Method: Checks if currentToken is valid and not expired. If expired or null: calls buildAuthRequest() to construct HTTP request, sends it, parses response via parseApiResponse(). Caches the new token and expiry. Returns the valid token.
2	generateQRCode(invoice: Invoice, accessToken: String)	String	+	Invokes the VietQR endpoint to generate a QR code image. Parameters: • invoice: Invoice — invoice data for QR content. • accessToken: String — valid API token. Method: Calls buildQRCodeRequest(invoice, accessToken) to construct HTTP request. Sends the request and parses response. Returns Base64-encoded QR image string.
3	paymentCallback(transactionResult: String)	void	+	The endpoint mapping handler for incoming VietQR webhook callbacks. Parameters: • transactionResult: String — raw JSON payload from VietQR. Method: Delegates processing to PayThroughVietQRController.handlePaymentCallback(transactionResult).

4	buildAuthRequest()	HttpRequest	-	Private utility. Creates the authentication HTTP request object with API Key headers.
5	buildQRCodeRequest(invoice: Invoice, token: String)	HttpRequest	-	Private utility. Creates the QR generation HTTP request with invoice payload. Parameters: • invoice: Invoice — source data. • token: String — auth token for header.
6	parseApiResponse(response: HttpResponse)	String	-	Private utility. Converts and validates the HTTP response body. Parameters: • response: HttpResponse — raw API response.

1.11 PayPalBoundary

Stereotype: <<*boundary*>> (System Interface)
System Interface handling PayPal Sandbox API integration. Authentication via OAuth2 client_credentials grant. Maps to NestJS Provider paypal.provider.ts.

1.11.1 Attributes

#	Name	Type	Default	Vis.	Description
1	apiBaseUrl	String	"https://api.sandbox.paypal.com"	-	Sandbox URL for PayPal API.
2	clientId	String	env.PAYPAL_CLIENT_ID	-	PayPal Business OAuth2 Client ID.
3	clientSecret	String	env.PAYPAL_CLIENT_SECRET	-	PayPal Business OAuth2 Secret.
4	currentToken	String	null	-	Cached OAuth2 Bearer Token.
5	tokenExpiry	Date	null	-	Expiry time of the cached token.

1.11.2 Operations

#	Name	Return Type	Vis.	Details
1	getAccessToken()	String	+	Performs OAuth2 client_credentials grant to acquire a Bearer Token. Method: Checks if currentToken is valid and not expired. If expired: calls buildAuthRequest() with Basic Auth (clientId:clientSecret), sends request. Parses response, caches token and expiry. Returns the valid Bearer token.
2	processPayment(cardInfo: CreditCard, amount: double, accessToken: String)	boolean	+	Posts card info to PayPal checkout captures endpoint. Returns synchronous success status. Parameters: • cardInfo: CreditCard — card details to charge. • amount: double — payment amount. • accessToken: String — valid Bearer token. Method: Calls buildPaymentRequest(cardInfo, amount, accessToken) to construct HTTP request. Sends request to PayPal Orders API. Parses response for success/failure status. Returns true if capture status is COMPLETED.
3	refundPayment(transactionId: String, amount: double)	boolean	+	Posts a refund request for an existing captured payment. **Parameters:** • transactionId: String — the PayPal capture ID to refund. • amount: double — the amount to refund.

				Method: Acquires a fresh token via getAccessToken(). Sends POST to PayPal Refund endpoint. Returns true if refund status is COMPLETED.
4	buildAuthRequest()	HttpRequest	-	Private utility. Builds the OAuth2 HTTP request using Basic Auth.
5	buildPaymentRequest(cardInfo: CreditCard, amount: double, token: String)	HttpRequest	-	Private utility. Builds the payment POST capture HTTP request. **Parameters:** • cardInfo: CreditCard — card details payload. • amount: double — charge amount. • token: String — Bearer token for Authorization header.
6	parseApiResponse(response: HttpResponse)	String	-	Private utility. Safely parses PayPal's JSON response body. **Parameters:** • response: HttpResponse — raw HTTP response from PayPal.

2.1 PlaceOrderController

Stereotype: <<control>>

Orchestrates the entire Place Order flow. Manages stock validation, delivery info submission, shipping fee calculation, payment delegation, invoice/order persistence, and email notification.

2.1.1 Attributes

#	Name	Type	Default	Visibility	Description
1	order	Order		-	The Order object being built through the flow; persisted at end of successful payment

2.1.2 Operations

#	Name	Return Type	Vis.	Details
1	placeOrder(cart: Cart)	void	+	Entry point for Place Order use case. Validates stock for all cart items. On success, creates a new Order(cart) and shows delivery form.
2	validateStock(cart: Cart)	boolean	-	Iterates over each CartItem and calls product.isAvailable(quantity). Collects all insufficient items. Returns false and throws InvalidQuantityException if any item fails.
3	submitDeliveryInfo(deliveryInfo: DeliveryInfo)	void	+	Validates delivery info; on success calls order.setDeliveryInfo(deliveryInfo), then order.calculateShippingFee(), then creates and shows invoice.
4	checkDeliveryInfoValidity(deliveryInfo: DeliveryInfo)	void	+	Checks all required fields (receiverName, email, phone, province, address) are non-empty. Validates email format (regex) and phone number format (10 digits, leading 0). Returns false with field-level errors if any fail.
5	calculateShippingFee(deliveryInfo: DeliveryInfo, cart: Cart)	double	+	Applies shipping rules: free shipping up to 25,000 VND if subtotal > 100,000 VND; base fee by location (HN/HCM: 22,000 first 3kg; elsewhere: 30,000 first 0.5kg); +2,500 VND per 0.5kg beyond threshold.
6	payOrder(order: Order)	void	+	Delegates to PayOrderController.payOrder(order). On success: calls createInvoice(), invoice.saveInvoice(), order.saveOrder(), cart.empty(), sendEmailToCustomer(invoice), then shows SuccessfulScreen.
7	createInvoice(order: Order)	Invoice	-	Constructs an Invoice with totalCostExclVAT from cart subtotal, totalCostInclVAT (×1.1), deliveryFee, and totalAmount; sets createdAt = now.
8	sendEmailToCustomer(invoice: Invoice)	void	+	Sends an email to deliveryInfo.email containing the invoice details and payment

				transaction information. Uses a notification/email service.

2.1.3 Method — calculateShippingFee(deliveryInfo, cart)

1. subtotal = cart.calculateSubTotal(); totalWeight = cart.getTotalWeight()
2. Determine base fee: if deliveryInfo.isHanoiOrHCMC() → threshold = 3 kg, base = 22,000; else → threshold = 0.5 kg, base = 30,000.
3. If totalWeight > threshold, add ⌈(totalWeight − threshold) / 0.5⌉ × 2,500 to base fee.
4. If subtotal > 100,000: shippingFee = Math.max(0, base − Math.min(base, 25,000)).
5. Else: shippingFee = base. Return shippingFee.

2.1.4 Exceptions

InvalidQuantityException — thrown by placeOrder() when one or more cart items exceed available stock.
InvalidDeliveryInfoException — thrown by submitDeliveryInfo() when delivery info validation fails.

2.2 Product Controller

Stereotype: <<control>>

Coordinates the Create Product, Update Product, and Delete Product use cases. It receives requests from product management screens, validates product information, delegates processing to the corresponding entity logic, records product history, and returns the result to the UI.

2.2.1 Attributes

#	Name	Type	Default	Visibility	Description
1	selectedProduct	Product	null	-	The product currently selected for update or delete operations.
2	ProductInfo	ProductInfo	null	-	Product information currently being processed.
3	actionResult	boolean	false	-	Result of the latest create/update/delete action.

2.2.2 Operations

#	Name	Return Type	Vis.	Details
1	createProduct(productInfo: ProductInfo)	Product	+	Creates a new Product object from validated product information. **Parameters:** • productInfo: ProductInfo — validated information entered by the product manager. **Method:** Validates the input, creates a new Product, persists it, creates a ProductHistory record, sets actionResult = true, and returns the created Product.
2	updateProduct(product: Product, productInfo: ProductInfo)	void	+	Updates an existing product with the given information. **Parameters:** • product: Product — the selected product to update. • productInfo: ProductInfo — the new validated product information. **Method:** Validates the input, calls product.updateProductInfo(productInfo), saves the updated product, creates a ProductHistory record, and sets actionResult = true.
3	deleteProduct(product: Product)	void	+	Deletes or deactivates a product depending on current stock quantity. **Parameters:** • product: Product — the selected product to delete. **Method:** Checks product.checkStock(). If quantity == 0, deletes the product physically. Otherwise, changes product status to DEACTIVATED. Creates a ProductHistory record and sets actionResult = true.

4	validateProductInfo(productInfo: ProductInfo)	boolean	+	Validates the product information submitted by the product manager. **Parameters:** • productInfo: ProductInfo — input product information to validate. **Method:** Calls productInfo.validateProductInfo() and returns true if valid; otherwise returns false.
5	returnResult(result: boolean)	void	+	Returns the action result to the corresponding boundary screen. **Parameters:** • result: boolean — processing result to be shown on UI. **Method:** Sets actionResult = result and triggers the
6	getProductStatus()	String	+	Returns the status of the currently selected product. Method: Returns selectedProduct.status as string.

2.2.3 Method

createProduct(productInfo)
• What: Creates a new product from validated input.
• How:
    o Calls validateProductInfo(productInfo)
    o Creates Product from productInfo
    o Saves Product
    o Creates ProductHistory with action = "CREATE"
    o Returns success result
• Use attributes: productInfo, actionResult
• Use relationships: interacts with Product, ProductHistory, ProductInfo

updateProduct(product, productInfo)
• What: Updates product information.
• How:
    o Validates productInfo
    o Calls product.updateProductInfo(productInfo)
    o Saves updated product
    o Creates ProductHistory with action = "UPDATE"
• Use attributes: selectedProduct, productInfo, actionResult
• Use relationships: interacts with Product, ProductHistory, ProductInfo

deleteProduct(product)
• What: Removes or deactivates a product.
• How:
    o Calls product.checkStock()
    o If stock = 0, deletes product
    o Else changes status to DEACTIVATED
    o Creates ProductHistory with action = "DELETE" or "DEACTIVATE"
• Use attributes: selectedProduct, actionResult
• Use relationships: interacts with Product, ProductHistory

validateProductInfo(productInfo)
• What: Validates product input data.
• How:
    o Calls productInfo.validateProductInfo()
    o Returns validation result
• Use attributes: productInfo
• Use relationships: interacts with ProductInfo

2.2.4 State Machine

2.3 PayOrderController

Stereotype: <<*control*>> | Package: controls

The entry point controller for the Pay Order use case. Routes the execution flow to the appropriate payment gateway based on the user's selected payment method (VietQR or Credit Card).

2.3.1 Attributes

#	Name	Type	Default	Vis.	Description
1	*invoice*	Invoice	null	-	The invoice to be paid.
2	*paymentResult*	boolean	false	-	State flag to store the payment outcome.
3	*paymentMethod*	PaymentMethod	—	-	The payment method selected by the user.

2.3.2 Operations

#	Name	Return Type	Vis.	Details
1	*PayOrder(invoice: Invoice)*	void	+	The main entry point initiating payment. **Parameters:** • invoice: Invoice — the invoice object requiring payment. **Method:** 1. Store invoice into this.invoice. 2. Determine paymentMethod selected by the Customer on the UI. 3. If paymentMethod == VIETQR: call initiateVietQRPayment(invoice). 4. Else if paymentMethod == CREDIT_CARD: call initiateCreditCardPayment(invoice). 5. Wait for result — returnPaymentResult() is called by the sub-controller.
2	*returnPaymentResult(paymentResult: boolean)*	void	+	Sends the final payment status back to the PlaceOrderController. **Parameters:** • paymentResult: boolean — true if payment succeeded, false otherwise.
3	*initiateVietQRPayment(invoice: Invoice)*	void	+	Initiates the asynchronous VietQR payment flow. **Parameters:** • invoice: Invoice — the invoice to pay. **Method:** 1. qrCode = PayThroughVietQRController.generateQRCode(invoice). 2. If qrCode != null: call VietQRPaymentScreen.displayQRCode(invoice, qrCode) and wait for async callback from VietQR. 3. Else: call VietQRPaymentScreen.showPaymentError("Cannot generate QR code").

4	*initiateCreditCardPayment(invoice: Invoice)*	void	+	Initiates the synchronous Credit Card payment flow. **Parameters:** • invoice: Invoice — the invoice to pay. **Method:** 1. Call CreditCardPaymentForm.displayPaymentForm(invoice). 2. Wait for Customer to input and submit cardInfo. 3. success = PayThroughCreditCardController.processPayment(cardInfo, invoice). 4. If success: call CreditCardPaymentForm.showPaymentResult(true) then returnPaymentResult(true). 5. Else: call CreditCardPaymentForm.showPaymentResult(false) then returnPaymentResult(false).
5	*getPaymentStatus()*	String	+	Retrieves the current generic payment status.

2.4 PayThroughVietQRController

Stereotype: <<*control*>> | Package: controls

Handles business logic exclusively for the asynchronous VietQR payment flow. Manages two phases: (1) generating the QR code and (2) handling the webhook callback from VietQR.

2.4.1 Attributes

#	Name	Type	Default	Vis.	Description
1	*accessToken*	String	null	-	Temporarily stored API token for VietQR calls.
2	*callbackVerified*	boolean	false	-	Verification status of the received webhook data integrity.
3	*transactionData*	String	null	-	The raw webhook payload data received from VietQR.

2.4.2 Operations

#	Name	Return Type	Vis.	Details
1	*generateQRCode(invoice: Invoice)*	String	+	Phase 1: Generates the QR code to be displayed on screen. **Parameters:** • invoice: Invoice — the invoice to generate QR for. **Exceptions:** • PaymentGatewayException — if token request fails. • QRCodeGenerationException — if QR generation fails. **Method:** 1. accessToken = requestAccessToken() — calls VietQRBoundary.getAccessToken(). 2. If token fails — throw PaymentGatewayException. 3. qrCode = requestQRCodeGeneration(invoice, accessToken) — calls VietQRBoundary.generateQRCode(). 4. If QR fails — throw QRCodeGenerationException. 5. Return qrCode.
2	*handlePaymentCallback(transactionResult: String)*	void	+	Phase 2: Processes the webhook callback from VietQR after customer payment. **Parameters:** • transactionResult: String — the raw JSON callback payload. **Method:** 1. callbackVerified = verifyCallbackData(transactionResult). 2. If callbackVerified == true:

				2.1. txn = createPaymentTransaction(transactionResult). 2.2. txn.save(). 2.3. PayOrderController.returnPaymentResult(true). 3. Else: 3.1. Log the error. 3.2. PayOrderController.returnPaymentResult(false).
3	*verifyCallbackData(transactionResult: String)*	boolean	+	Verifies the callback data integrity (signature, format, amount match). **Parameters:** • transactionResult: String — raw callback payload to verify.
4	*requestAccessToken()*	String	-	Private. Requests authentication token from VietQRBoundary.getAccessToken(). **Method:** Delegates to VietQRBoundary.getAccessToken() and caches the result.
5	*requestQRCodeGeneration(invoice: Invoice, accessToken: String)*	String	-	Private. Requests QR image generation from VietQRBoundary.generateQRCode(). **Parameters:** • invoice: Invoice — invoice data for QR content. • accessToken: String — valid API token.
6	*createPaymentTransaction(transactionResult: String)*	PaymentTransaction	-	Private. Instantiates a PaymentTransaction entity from parsed webhook data. **Parameters:** • transactionResult: String — raw callback data to parse. **Method:** Parses transactionResult JSON, extracts amount/orderId/content. Creates PaymentTransaction with method = VIETQR, status = SUCCESS, rawCallbackData = transactionResult.

2.5. PayThroughCreditCardController

Stereotype: <<*control*>> | Package: controls

Handles the business logic for verifying and processing synchronous payments via the PayPal gateway. Provides automatic refund capabilities for cancelled/rejected orders.

2.5.1 Attributes

#	Name	Type	Default	Vis.	Description
1	*creditCard*	CreditCard	null	-	Reference to the card info submitted by the customer on the form.
2	*accessToken*	String	null	-	OAuth2 Bearer Token used to connect to PayPal.

2.5.2 Operations

#	Name	Return Type	Vis.	Details
1	*processPayment(cardInfo: CreditCard, invoice: Invoice)*	boolean	+	The primary handler for the Credit Card payment logic. **Parameters:** • cardInfo: CreditCard — card details from the form. • invoice: Invoice — the invoice to pay. **Method:** 1. isValid = validateCardInfo(cardInfo). 2. If !isValid — return false. 3. accessToken = requestAccessToken(). 4. amount = invoice.getGrandTotal(). 5. success = sendPaymentRequest(cardInfo, amount, accessToken). 6. txn = createPaymentTransaction(cardInfo, invoice, success). 7. txn.save(). 8. If success: invoice.setPaymentTransaction(txn). 9. Return success.
2	*validateCardInfo(card: CreditCard)*	boolean	+	Validates the card using Luhn algorithm and expiry check. **Parameters:** • card: CreditCard — card to validate. **Method:** Delegates to card.validateCard().
3	*refundPayment(transaction: PaymentTransaction)*	boolean	+	Executes automatic refund logic for Cancelled/Rejected orders. **Parameters:** • transaction: PaymentTransaction — the original successful transaction to refund. **Method:** 1. accessToken = requestAccessToken(). 2. success = PayPalBoundary.refundPayment(transaction.paypalOrderId, transaction.amount).

				3. If success: 3.1. transaction.updateStatus(REFUNDED). 3.2. Set transaction.refundId, refundDate, refundAmount. 3.3. transaction.save(). 4. Return success.
4	*requestAccessToken()*	String	-	Private. Requests OAuth2 token from PayPalBoundary.getAccessToken().
5	*sendPaymentRequest(cardInfo: CreditCard, amount: double, accessToken: String)*	boolean	-	Private. Sends checkout payment payload to PayPalBoundary.processPayment(). **Parameters:** • cardInfo: CreditCard — card details. • amount: double — payment amount. • accessToken: String — valid Bearer token.
6	*createPaymentTransaction(cardInfo: CreditCard, invoice: Invoice, success: boolean)*	PaymentTransaction	-	Private. Instantiates a PaymentTransaction entity and saves to database. **Parameters:** • cardInfo: CreditCard — for extracting cardLastFour. • invoice: Invoice — for orderId and amount. • success: boolean — payment result. **Method:** Creates PaymentTransaction with method = CREDIT_CARD. Sets status = SUCCESS or FAILED based on success. Sets cardLastFour = cardInfo.maskCardNumber().

3.1 CartItem

Stereotype: <<entity>>

Holds a product reference and the quantity the customer intends to purchase. Responsible for computing its own subtotal and weight contribution.

3.1.1 Attributes

#	Name	Type	Default	Visibility	Description
1	product	Product		-	The product in this cart slot
2	quantity	int	1	-	Number of units of this product; must be >= 1

3.1.2 Operations

#	Name	Return Type	Vis.	Details
1	CartItem(product: Product, quantity: int)	CartItem	+	Constructor. Validates quantity ≥ 1.
2	getSubtotal()	double	+	Returns product.cost × quantity (excl. VAT).
3	getWeight()	double	+	Returns product.weight × quantity in kilograms; used for shipping fee calculation.

3.2 Cart

Stereotype: <<entity>>

Holds the collection of CartItems representing the customer's current shopping session. Provides aggregate calculations for subtotal and total weight.

3.2.1 Attributes

#	Name	Type	Default	Visibility	Description
1	items	CartItem[]	[]	-	Ordered list of cart item entries; no duplicate productIds allowed

3.2.2 Operations

#	Name	Return Type	Vis.	Details
1	calculateSubTotal()	double	+	Returns the sum of item.getSubtotal() for all items; excludes VAT.
2	getTotalWeight()	double	+	Returns the sum of item.getWeight() for all items; used by shipping fee calculation.
3	emptyCart()	void	+	Clears items array after a successful order is placed.
4	addItem(product: Product, quantity: number)	void	+	Creates a new CartItem or increments quantity if product already exists.
5	removeItem(productId: string)	void	+	Removes the item with the given productId from the cart.
6	updateQuantity(productId: string, quantity: number)	void	+	Updates the quantity of a CartItem; removes item if quantity set to 0.
7	getItem(productId: string)	CartItem	+	Returns the CartItem for a given productId, or undefined if not in cart.

3.3 Product

Stereotype: <<entity>>

Represents a media product managed in AIMS. It stores the core information of a product and supports product-related business actions such as updating information, checking whether the product can be physically deleted, and changing its business status.

3.3.1 Attributes

#	Name	Type	Default	Visibility	Description
1	productId	String	auto-generated	-	Unique identifier of the product.
2	productName	String	“”	-	Name of the product.
3	category	String	“”	-	Category/type of the product, such as Book, CD, or DVD.
4	price	double	0.0	-	Current selling price excluding 10% VAT; must be 30–150% of originalValue
5	originalValue	double		-	Original/base price of the product
6	quantity	int	0	-	Current stock quantity of the product
7	description	String	“”	-	Description of the product shown to users.
8	barcode	String		-	Product barcode string
9	imgUrl	String	“”	-	URL to product image displayed in cart and screens
10	status	String	ACTIVE	-	Current business status of the product (e.g. ACTIVE, DEACTIVATED, DELETED).
11	createdAt	LocalDateTime	now()	-	Timestamp when the product was first created.
12	updatedAt	LocalDateTime	now()	-	Timestamp when the product was last updated.

3.3.2 Operations

	Name	Return Type	Vis.	Details
1	Product(productName: String, category: String, price: double, quantity: int, description: String)	—	+	Constructor. Creates a new Product object with the given initial information. **Parameters:** • productName: String — product name. • category: String — product category. • price: double — initial product price. • quantity: int — initial stock quantity. • description: String — product description. **Method:** Generates productId automatically. Sets all basic attributes from input parameters. Sets status = ACTIVE. Sets createdAt = LocalDateTime.now() and updatedAt = LocalDateTime.now().

2	updateInfo(productName: String, category: String, price: double, quantity: int, description: String)	void	+	Updates editable product information. **Parameters:** • productName: String — new product name. • category: String — new category. • price: double — new selling price. • quantity: int — new stock quantity. • description: String — new description. **Method:** Replaces the current editable fields with the new values. Updates updatedAt = LocalDateTime.now().
3	changeStatus(status: ProductStatus)	void	+	Changes the business status of the product. **Parameters:** • status: ProductStatus — target status value. **Method:** Sets this.status to the given value and updates updatedAt.
4	getQuantityInStock(id: string)	double	+	Looks up and returns quantityInStock for the given product ID. Called by controller during stock validation.
5	isAvailable(requestedQty: number)	boolean	+	Returns isActive && quantityInStock >= requestedQty. Used by PlaceOrderController.validateStock().
6	getCostWithVAT()	double	+	Returns cost × 1.1. Used when computing invoice totals including VAT.
7	isDeletable()	boolean	+	Checks whether the product is allowed to be physically deleted. **Method:** Returns true if quantity == 0. Returns false otherwise.
8	deactivate()	void	+	Marks the product as deactivated instead of deleting it physically. **Method:** Sets status = DEACTIVATED. Updates updatedAt = LocalDateTime.now().

3.3.3 Method

The Product class is responsible for maintaining the core state of a product object. Its operations are used by ProductController during the Create Product, Update Product, and Delete Product use cases.

- The constructor Product(...) is called when a product manager creates a new product.
- updateInfo(...) is called when valid updated product data is submitted.
- isDeletable() is used during the Delete Product use case to determine whether the product can be removed physically or should be deactivated instead.
- deactivate() is used when the product still has stock quantity greater than zero.
- changeStatus(...) provides a general mechanism for status transitions when needed.

The Product object collaborates mainly with:
- ProductController, which coordinates the use cases
- ProductHistory, which records product-related actions

3.3.4 State Machine

The Product object transitions through the following states:

• ACTIVE — product is available in the system and can be updated or viewed normally.
• DEACTIVATED — product is no longer active for business use, but still remains in the system.

• DELETED — product is removed from active storage/business usage.

Typical transitions:
• createProduct → ACTIVE
• updateProduct → ACTIVE (state unchanged, data updated)
• deactivateProduct [quantity > 0] → DEACTIVATED
• deleteProduct [quantity = 0] → DELETED

3.4 ProductHistory

Stereotype: <<entity>>

Represents a history record of product-related actions. It is used to track creation, update, deletion, and deactivation activities performed on a product.

3.4.1 Attributes

#	Name	Type	Default	Visibility	Description
1	historyId	String	auto-generated	-	Unique identifier of the history record.
2	productId	String	null	-	Identifier of the related product.
3	action	String	“”	-	Action performed on the product, such as CREATE, UPDATE, DELETE, or DEACTIVATE.
4	actionTime	Date	now()	-	Timestamp when the action was performed.

3.4.2 Operations

	Name	Return Type	Vis.	Details
1	ProductHistory(productId: String, action: String, actionTime: Date)	—	+	Constructor. Creates a new history record for a product action. **Parameters:** • productId: String — identifier of the related product. • action: String — action performed. • actionTime: Date — action timestamp. **Method:** Generates historyId automatically. Sets productId, action, and actionTime.
2	save()	void	+	Persists the history record. **Method:** Saves the current ProductHistory object into storage.
3	getHistoryInfo()	String	+	Returns a human-readable summary of the history record. **Method:** Returns a string containing productId, action, and actionTime.
4	recordAction(action: String)	void	+	Updates the action of the current history record. **Parameters:** • action: String — action value to record. **Method:** Sets this.action = action and updates actionTime = current time.

3.4.3 Method

ProductHistory(productId, action, actionTime)
• What: Creates a history record.
• How:

    - Generates historyId
    - Stores productId
    - Stores action and actionTime
- Use attributes: historyId, productId, action, actionTime

save()
- What: Persists history data.
- How:
    - Saves the current ProductHistory object
- Use attributes: all attributes

getHistoryInfo()
- What: Returns history summary information.
- How:
    - Builds and returns a formatted string from productId, action, and actionTime
- Use attributes: productId, action, actionTime

recordAction(action)
- What: Records or updates a product-related action.
- How:
    - Sets action = input value
    - Sets actionTime = current time
- Use attributes: action, actionTime

3.4.4 State Machine

3.5 Order

Stereotype: <<entity>>

Represents a customer's order including its total amount, status, and creation details.

3.5.1 Attributes

#	Name	Type	Default	Visibility	Description
1	orderId	String	“”	-	Unique identifier for the order.
2	cart	Cart		-	Snapshot of the cart at the time of order placement
3	deliveryInfo	DeliveryInfo		-	Delivery address and recipient detail; set after delivery form is submitted
4	Invoice	Invoice		-	Invoice associated with this order; set after delivery info is confirmed
5	totalAmount	double	0	-	Total amount that customer pays: total incl. VAT + shippingFee
6	status	OrderStatus	“”	-	Current status of the order.
7	shippingFee	double	0	-	Shipping fee applied to the order.
8	createdAt	Date	null	-	The date when the order is created.

3.5.2 Operations

#	Name	Return Type	Vis.	Details
1	Order(cart: Cart)	Order	+	Constructor. Sets orderId, createdAt, and cart. Initial status = PENDING.
2	setDeliveryInfo(deliveryInfo: DeliveryInfo)	void	+	Sets this.deliveryInfo and immediately triggers calculateShippingFee() and updateTotalAmount().

3	calculateShippingFee()	double	+	Delegates to the shipping fee logic (applied from PlaceOrderController). Updates and returns this.shippingFee.
4	updateTotalAmount()	void	-	Recomputes totalAmount = cart.calculateSubTotal() × 1.1 + shippingFee.
5	getOrderId()	String	+	Returns orderId.
6	getTotalAmount()	double	+	Returns totalAmount.
7	updateStatus(newStatus: OrderStatus)	void	+	Updates order status to newStatus
8	saveOrder()	void	+	Saves order to database.

3.5.3 State Machine

States:

- PENDING
- APPROVED
- REJECTED
- CANCELLED

3.6 Invoice

Stereotype: <<entity>>

Represents the billing information generated for an order.

3.6.1 Attributes

#	Name	Type	Default	Visibility	Description
1	invoiceId	String	“”	-	Unique identifier for the invoice.
2	totalCostExclVAT	double		-	Sum of all (product.cost × quantity) across cart items, before VAT
3	totalCostInclVAT	double		-	Total cost including 10% VAT: totalCostExclVAT × 1.1
4	shippingFee	double		-	Shipping fee (not subject to tax); copied from order at invoice creation
5	issueDate	Date	null	-	Date when the invoice is issued.
6	totalAmount	double	0	-	Final amount customer pays: totalCostInclVAT + deliveryFee
7	status	String	“”	-	Current status of the invoice.

3.6.2 Operations

#	Name	Return Type	Vis.	Details
1	Invoice()	void	+	Constructor. Initializes invoiceId, amount, issueDate.
2	saveInvoice()	String	+	Persists the invoice record to the database. Called after successful payment.

3	setPaymentTransaction(txn: PaymentTransaction)	double	+	Sets transaction.
4	calculateTotalAmount()	double	+	Recomputes and returns totalCostInclVAT + deliveryFee; called whenever fee changes.

3.6.3 State Machine

States:

- CREATED
- UPDATED
- SAVED

Transitions:

- CREATED → UPDATED (setPaymentTransaction)
- UPDATED → SAVED (saveInvoice)

3.7 CreditCard

Stereotype: <<entity>>

Represents the credit card information used for payment.

3.7.1 Attributes

#	Name	Type	Default	Visibility	Description
1	cardNumber	String	“”	-	The credit card number.
2	expiryDate	String	“”	-	Expiration date of the card.
3	cvv	String	“”	-	Security code of the card.
4	cardHolderName	String	“”	-	Name of the cardholder.

3.7.2 Operations

#	Name	Return Type	Vis.	Details
1	validateCard()	boolean	+	Validates card.
2	isExpired()	boolean	+	Checks expiration.
3	maskCardNumber()	String	+	Returns masked card number.

3.7.3 Method

validateCard()

- What: Validates card.

- How:
    - Checks card number, expiry, CVV
- Use attributes: cardNumber, expiryDate, cvv

isExpired()

- What: Checks expiration.
- How:
    - Compares expiryDate with current date
- Use attributes: expiryDate

maskCardNumber()

- What: Masks card number.
- How:
    - Hides all but last digits
- Use attributes: cardNumber

3.7.4 State Machine

States:

- VALID
- INVALID
- EXPIRED

Transitions:

- VALID → EXPIRED (time passes)
- VALID → INVALID (invalid input)

3.8 DeliveryInfo

Stereotype: <<entity>>

Holds the recipient's delivery details. Provides validation logic and location checks needed for shipping fee calculation.

3.8.1 Attributes

#	Name	Type	Default	Visibility	Description
1	receiverName	String	“”	-	Ordered list of cart item entries; no duplicate productIds allowed
2	email	String	“”	-	Recipient email for invoice delivery; required; must be valid email format
3	phoneNumber	String	“”	-	Recipient phone number; required; 10-digit Vietnamese format
4	province	String	“”	-	Delivery province/city; required; determines shipping fee tier
5	address	String	“”	-	Street address; required
6	ward	String	“”	-	Ward/district; required
7	note	String	“”	-	Delivery instructions; optional (e.g., "Leave at the door")

3.8.2 Operations

#	Name	Return Type	Vis.	Details
1	submitDeliveryInfo()	void	+	Persists validated delivery info to the backend. Called by the controller after validation passes.
2	validate()	boolean	+	Validates all required fields are non-empty; checks email format (RFC 5322 regex) and phone format (10 digits, starts with 0). Returns false if any check fails.
3	isHanoiOrHCMC()	boolean	+	Returns true if province matches "Hà Nội" or "Hồ Chí Minh". Used by PlaceOrderController.calculateShippingFee() to choose the correct base rate.

3.9. PaymentTransaction

Stereotype: <<*entity*>>
Records payment transaction data. Stores information for both VietQR and CreditCard payment methods, mapped to the payment_transactions table in the database.

3.9.1 Attributes

#	Name	Type	Default	Vis.	Description
1	transactionId	String	auto	-	Automatically generated transaction ID.
2	orderId	String	—	-	The associated order ID.
3	amount	double	0.0	-	The payment amount.
4	method	PaymentMethod	—	-	Payment method used (VIETQR or CREDIT_CARD).
5	status	TransactionStatus	PENDING	-	Current state of the transaction.
6	transactionContent	String	""	-	Description of the transaction.
7	transactionDate	Date	now()	-	Timestamp when the transaction occurs.
8	createdAt	Date	now()	-	Timestamp when the record is created in the database.
9	rawCallbackData	String	null	-	(VietQR only) Raw JSON webhook payload from VietQR callback.
10	paypalOrderId	String	null	-	(CreditCard only) The Order ID on the PayPal system.
11	cardLastFour	String	null	-	(CreditCard only) Last 4 digits of the card for tracking.
12	refundId	String	null	-	(CreditCard only) Refund reference ID returned by PayPal API.
13	refundDate	Date	null	-	(CreditCard only) Timestamp of the refund.
14	refundAmount	double	0.0	-	(CreditCard only) Amount refunded.

3.9.2 Operations

#	Name	Return Type	Vis.	Details
1	paymentTransaction()	—	+	Constructor. Initializes an empty PaymentTransaction object with default values. **Method:** Initializes all fields to their specified default values.
2	save()	void	+	Persists the transaction to the database. **Method:** Performs a CREATE or UPDATE operation on the payment_transactions table.

3	updateStatus(newStatus: TransactionStatus)	void	+	Updates the transaction status. **Parameters:** • newStatus: TransactionStatus — the target status (e.g., SUCCESS, FAILED, REFUNDED). **Method:** Assigns newStatus to this.status. Calls this.save() to persist the change.
4	getTransactionInfo()	TransactionInfo	+	Retrieves a summarized view of the transaction for display purposes. **Method:** Returns a TransactionInfo object containing transactionId, amount, method, status, and transactionDate.
5	parseResponseString(response: string)	void	+	Parses the raw JSON/string response from the payment gateway and populates transactionId, status, time, and errorMessage.